

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ  
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ  
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ  
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук  
Образовательная программа «Искусственный интеллект»

**Воспроизведение результатов статьи  
«RepVGG: Making VGG-style ConvNets Great Again»**

Отчет об исследовательском проекте

Выполнил студент:  
Власов Владислав Сергеевич

Москва 2026 г.

## АННОТАЦИЯ

Воспроизведены ключевые результаты статьи RepVGG [1] на CIFAR-10. Реализована архитектура RepVGG-A0 со структурной репараметризацией: при обучении блок содержит три параллельные ветки ( $3 \times 3$ ,  $1 \times 1$ , identity), при инференсе они сливаются в одну свёртку  $3 \times 3$  без потери точности. Проведены четыре группы экспериментов: верификация слияния весов, сравнение с ResNet-20 и VGG-16, ablation по веткам, влияние аугментации Cutout. Обучение и логирование выполнены на NVIDIA RTX 3070 с фиксацией метрик в Weights & Biases.

# Содержание

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| <b>1</b> | <b>Введение</b>                                       | <b>4</b>  |
| <b>2</b> | <b>Обзор архитектуры RepVGG</b>                       | <b>4</b>  |
| 2.1      | Тренировочный блок . . . . .                          | 4         |
| 2.2      | Репараметризация . . . . .                            | 4         |
| 2.3      | Конфигурация сети . . . . .                           | 5         |
| <b>3</b> | <b>Постановка задачи и адаптация</b>                  | <b>5</b>  |
| 3.1      | Датасет . . . . .                                     | 5         |
| 3.2      | Адаптация архитектуры . . . . .                       | 5         |
| 3.3      | Базовые модели . . . . .                              | 6         |
| <b>4</b> | <b>Экспериментальная методика</b>                     | <b>6</b>  |
| 4.1      | Параметры обучения . . . . .                          | 6         |
| 4.2      | Аугментации . . . . .                                 | 6         |
| 4.3      | Оборудование и воспроизводимость . . . . .            | 7         |
| <b>5</b> | <b>Эксперименты и результаты</b>                      | <b>7</b>  |
| 5.1      | Эксперимент 1: верификация репараметризации . . . . . | 7         |
| 5.2      | Эксперимент 2: сравнение архитектур . . . . .         | 7         |
| 5.3      | Эксперимент 3: ablation по веткам . . . . .           | 8         |
| 5.4      | Эксперимент 4: влияние Cutout . . . . .               | 9         |
| <b>6</b> | <b>Заключение</b>                                     | <b>10</b> |
|          | <b>Приложение</b>                                     | <b>13</b> |

# 1 Введение

Современные свёрточные архитектуры повышают точность за счёт усложнения топологии: skip-connections (ResNet), плотные связи (DenseNet), автоматически найденные графы (NAS). Побочный эффект - замедление инференса: разветвления снижают утилизацию GPU, увеличивают пиковый расход памяти и усложняют развёртывание.

Ding et al. [1] разделили тренировочную и инференс-архитектуру. При обучении каждый блок содержит три ветки, что расширяет пространство оптимизации. При деплое ветки сливаются в единственную свёртку  $3 \times 3$  через структурную репараметризацию, и модель становится plain-цепочкой вида conv-relu без разветвлений.

Оригинальная статья приводит результаты только для ImageNet, поэтому прямое сравнение наших чисел с авторскими невозможно. Цель работы - воспроизвести ключевую идею RepVGG (структурную репараметризацию) на CIFAR-10, проверить её корректность, оценить вклад отдельных веток и сравнить результат с ResNet-20 [2] и VGG-16 [3] в одинаковых условиях обучения.

## 2 Обзор архитектуры RepVGG

### 2.1 Тренировочный блок

При обучении RepVGG-блок суммирует три параллельные ветки:

- свёртка  $3 \times 3$  + BatchNorm,
- свёртка  $1 \times 1$  + BatchNorm,
- identity + BatchNorm (при  $C_{in} = C_{out}$ , stride = 1).

Выход до нелинейности:

$$y = \text{BN}_3(\text{Conv}_{3 \times 3}(x)) + \text{BN}_1(\text{Conv}_{1 \times 1}(x)) + \text{BN}_{id}(x)$$

Identity-ветка играет роль, аналогичную residual connection в ResNet, - облегчает распространение градиентов. Ветка  $1 \times 1$  расширяет множество достижимых функций при обучении, но исчезает при слиянии.

### 2.2 Репараметризация

Каждая из трёх веток линейна (до ReLU), поэтому их можно свернуть алгебраически.

Слияние свёртки с BatchNorm:

$$W'_{ij} = \frac{\gamma_i}{\sqrt{\sigma_i^2 + \epsilon}} \cdot W_{ij}, \quad b'_i = \beta_i - \frac{\gamma_i \mu_i}{\sqrt{\sigma_i^2 + \epsilon}}$$

где  $\gamma, \beta$  - обучаемые параметры BN, а  $\mu, \sigma^2$  - накопленные running-статистики.

Ядра  $1 \times 1$  и identity дополняются нулями до размера  $3 \times 3$ ; итоговый фильтр:

$$W_{\text{merged}} = W_{3 \times 3} + \text{pad}(W_{1 \times 1}) + \text{pad}(W_{\text{id}})$$

Результат - единственная Conv  $3 \times 3 + \text{bias}$ , за которой следует ReLU. Ни skip-connections, ни дополнительных BN-слоёв.

## 2.3 Конфигурация сети

RepVGG содержит пять стейджей; ширина каналов определяется множителями относительно базовых значений [64, 128, 256, 512]. Конфигурация A0 использует множители [0.75, 0.75, 0.75, 2.5], что даёт 7.84М параметров в тренировочном режиме. Детальная конфигурация приведена в таблице 1.

Таблица 1: Конфигурация RepVGG-A0 (адаптация под CIFAR)

| Стейдж                 | Блоков | Каналов | Stride | Выход          |
|------------------------|--------|---------|--------|----------------|
| 0                      | 1      | 48      | 1      | $32 \times 32$ |
| 1                      | 2      | 48      | 1      | $32 \times 32$ |
| 2                      | 4      | 96      | 2      | $16 \times 16$ |
| 3                      | 14     | 192     | 2      | $8 \times 8$   |
| 4                      | 1      | 1280    | 2      | $4 \times 4$   |
| GAP + Linear(1280, 10) |        |         |        |                |

## 3 Постановка задачи и адаптация

### 3.1 Датасет

CIFAR-10: 60 000 цветных изображений  $32 \times 32$ , 10 классов; разбиение 50 000 / 10 000 (train / test).

### 3.2 Адаптация архитектуры

Оригинальная RepVGG рассчитана на ImageNet ( $224 \times 224$ ). Для разрешения  $32 \times 32$  внесены изменения:

- первый conv со stride = 1 вместо 2,
- убран maxpool после первого слоя,
- оставлена конфигурация A0.

Пространственное разрешение меняется как  $32 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 4$ ; после последнего стейджа - Global Average Pooling и линейный классификатор.

### 3.3 Базовые модели

- **ResNet-20** - CIFAR-версия из [2]: три группы residual-блоков (16, 32, 64 канала), 0.27M параметров.
- **VGG-16-BN** - VGG с BatchNorm [3], адаптированный под CIFAR: GAP вместо трёх FC-слоёв, 14.72M параметров.

Размеры моделей различаются на два порядка (0.27M – 14.72M), поэтому сравнение отражает соотношение «точность / ёмкость», а не строгий бенчмарк при фиксированном бюджете.

## 4 Экспериментальная методика

### 4.1 Параметры обучения

Все модели обучены по единому протоколу без индивидуального тюнинга гиперпараметров - это сознательный выбор, позволяющий сравнивать архитектуры в одинаковых условиях, хотя и не гарантирующий оптимальный результат для каждой из них. Параметры протокола:

- оптимизатор: SGD, momentum = 0.9, weight decay =  $10^{-4}$ ;
- начальный learning rate: 0.1;
- планировщик: cosine annealing до  $10^{-6}$ ;
- линейный warmup: 5 эпох;
- batch size: 128;
- mixed precision (FP16) через `torch.amp`;
- все запуски: 100 эпох.

### 4.2 Аугментации

Базовый набор: RandomCrop(32, padding = 4), RandomHorizontalFlip, нормализация по каналам (статистики CIFAR-10). В эксперименте 4 дополнительно применялся Cutout [4] с окном  $16 \times 16$ .

### 4.3 Оборудование и воспроизводимость

NVIDIA GeForce RTX 3070 (8 GB VRAM), PyTorch 2.6, CUDA 12.4. Все запуски проведены с `seed = 42`. Метрики логировались в Weights & Biases (проект RepVGG).

## 5 Эксперименты и результаты

### 5.1 Эксперимент 1: верификация репараметризации

Через обученную модель прогнан случайный тензор в тренировочном (три ветки) и deploy-режиме (одна свёртка). Поэлементная разница выходов не превышает  $\sim 10^{-6}$ , что укладывается в точность float32. Аккуратность на валидации совпадает.

*Замечание.* На GPU архитектуры Ampere (RTX 30xx) по умолчанию активен режим TF32, снижающий точность float32-операций. При включённом TF32 расхождение достигает  $\sim 10^{-3}$ , однако на классификационные метрики это не влияет. Для чистоты верификации TF32 отключался.

Таблица 2: RepVGG-A0: сравнение train- и deploy-режимов

| Режим                | Val Acc (%) | Параметры | FLOPs   | Инференс |
|----------------------|-------------|-----------|---------|----------|
| Train (multi-branch) | 93.88       | 7.84 M    | 984.8 M | 6.3 мс   |
| Deploy (single-path) | 93.88       | 7.04 M    | 883.0 M | 1.7 мс   |

Слияние веток даёт ускорение инференса в 3.7x. Число параметров снижается с 7.84M до 7.04M за счёт устранения отдельных BN-слоёв.

### 5.2 Эксперимент 2: сравнение архитектур

Три модели обучены на CIFAR-10 по 100 эпох при идентичном протоколе.

Таблица 3: Результаты на CIFAR-10 (100 эпох)

| Модель             | Val Acc (%) | Параметры (M) | FLOPs (M) | Инференс (мс) |
|--------------------|-------------|---------------|-----------|---------------|
| RepVGG-A0 (deploy) | 93.88       | 7.04          | 883.0     | 1.7           |
| ResNet-20          | 91.39       | 0.27          | 82.8      | 1.7           |
| VGG-16-BN          | 93.46       | 14.72         | 629.1     | 1.3           |

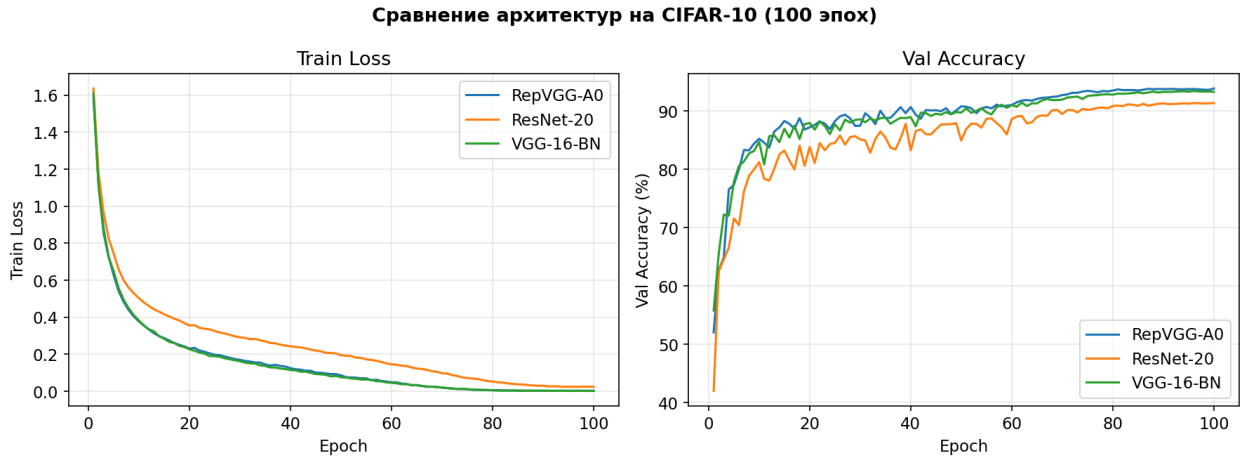


Рис. 1: Динамика обучения трёх моделей на CIFAR-10

RepVGG-A0 достигает 93.88%, опережая ResNet-20 на 2.5 п.п. и VGG-16 на 0.4 п.п. Deploy-версия RepVGG вдвое компактнее VGG-16 (7.04M против 14.72M) при меньшем числе FLOPs (883.0M против 629.1M). ResNet-20 при 0.27M параметров и 82.8M FLOPs показывает 91.39% — высокий результат для модели такого размера, но уступает RepVGG по ассурасу на 2.5 п.п.

Кривые обучения (рис. 1) показывают, что RepVGG-A0 и VGG-16 сходятся сравнимо быстро и достигают плато после  $\sim 60$  эпох, тогда как ResNet-20 выходит на свой максимум раньше, но на более низком уровне. У всех трёх моделей наблюдается разрыв между train loss и val ассурасу, характерный для обучения без сильной регуляризации: train loss продолжает снижаться, но val ассурасу перестаёт расти. Это согласуется с результатами эксперимента 4, где добавление Cutout дополнительно поднимает результат.

### 5.3 Эксперимент 3: ablation по веткам

Архитектура RepVGG удобна для ablation, поскольку три ветки тренировочного блока функционально независимы: identity обеспечивает градиентный поток (аналог skip-connection), ветка  $1 \times 1$  расширяет пространство оптимизации, а  $3 \times 3$  — основную ёмкость модели. Отключая ветки по одной, можно изолировать вклад каждой из них.

RepVGG-A0 обучена в четырёх конфигурациях по 100 эпох:

Таблица 4: Ablation: влияние отдельных веток (100 эпох)

| Конфигурация                                         | Val Acc (%) |
|------------------------------------------------------|-------------|
| $3 \times 3 + 1 \times 1 + \text{identity}$ (полная) | 93.88       |
| $3 \times 3 + 1 \times 1$ (без identity)             | 93.55       |
| $3 \times 3 + \text{identity}$ (без $1 \times 1$ )   | 93.29       |
| Только $3 \times 3$                                  | 93.38       |

Полная модель (93.88%) опережает все усечённые конфигурации: без identity – 93.55%,



без  $1 \times 1$  – 93.29%, только  $3 \times 3$  – 93.38%. Разница составляет 0.3–0.6 п.п. Наибольший вклад вносит ветка  $1 \times 1$  (её удаление снижает результат на 0.6 п.п.), что согласуется с гипотезой авторов о неявной регуляризации через расширение пространства оптимизации.

Конфигурация «только  $3 \times 3$ » (93.38%) показала результат чуть выше, чем «без  $1 \times 1$ » (93.29%). Разница 0.09 п.п. находится в пределах стохастического разброса одиночных запусков, поэтому мы не делаем из этого содержательных выводов.

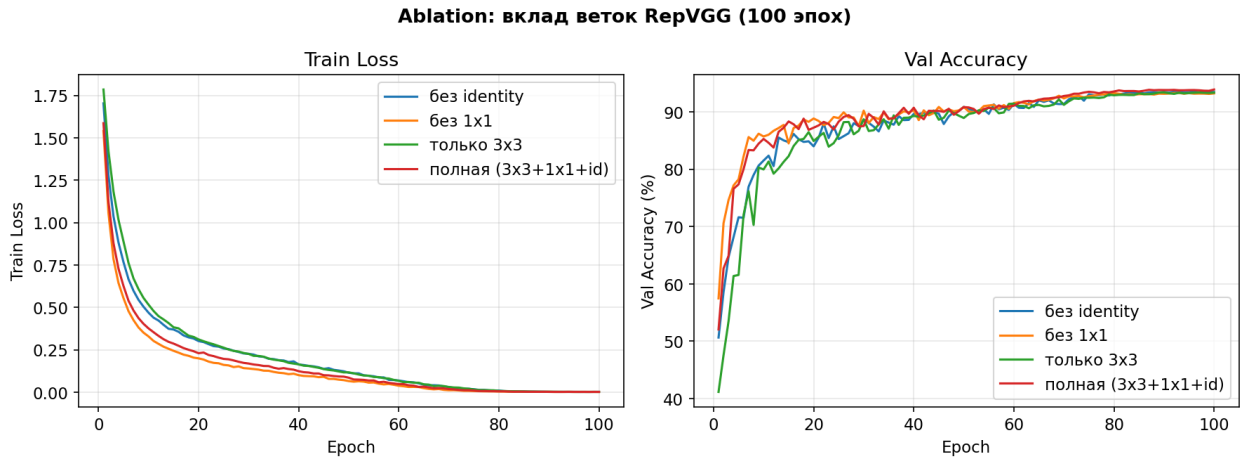


Рис. 2: Кривые обучения для разных конфигураций RepVGG (100 эпох)

## 5.4 Эксперимент 4: влияние Cutout

Cutout [4] случайно маскирует квадрат  $16 \times 16$  во входном изображении, вынуждая модель использовать пространственно распределённые признаки. RepVGG не содержит встроенных механизмов регуляризации типа Dropout между свёртками, поэтому внешняя регуляризация через аугментации особенно актуальна.

За 100 эпох RepVGG-A0 с Cutout достигает 94.90% против 93.88% без Cutout (+1 п.п.). Cutout даёт лучший результат среди всех конфигураций, подтверждая чувствительность plain-архитектур к внешней регуляризации.

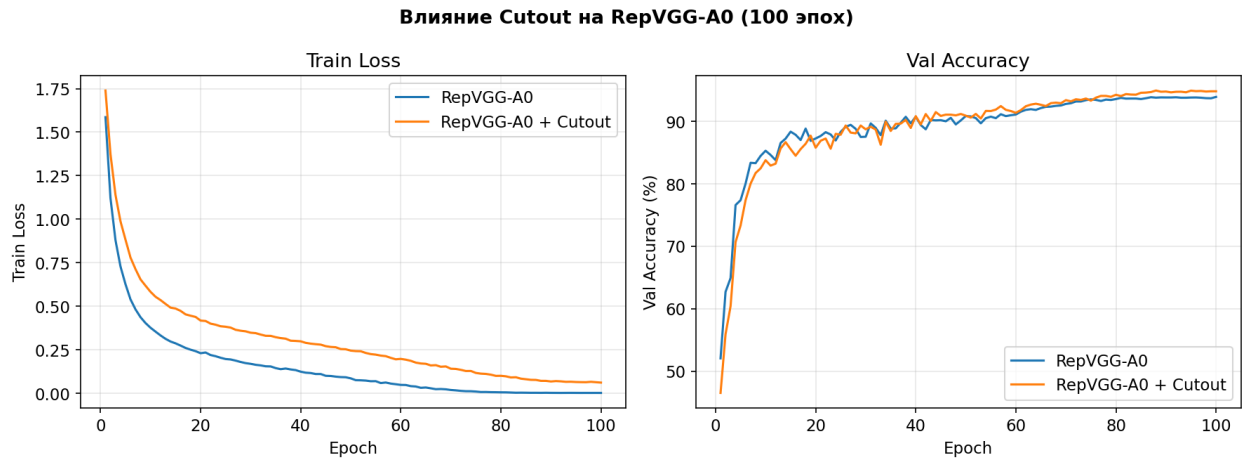


Рис. 3: Влияние Cutout на RepVGG-A0 (100 эпох)

## 6 Заключение

Репараметризация RepVGG корректна: разница выходов train- и deploy-версии составляет  $\sim 10^{-6}$ . Deploy-модель в 3.7x быстрее при полном сохранении ассурасу.

На CIFAR-10 RepVGG-A0 (93.88%) превосходит ResNet-20 (91.39%) и VGG-16 (93.46%), оставаясь вдвое компактнее VGG-16 в deploy-режиме.

Ablation подтвердил вклад каждой ветки: полная модель опережает усечённые на 0.3–0.6 п.п. Наибольший эффект даёт ветка  $1 \times 1$ . Cutout поднимает ассурасу до 94.90% (+1 п.п.), подтверждая чувствительность plain-архитектур к внешней регуляризации. Сводные результаты приведены в таблице 5.

Таблица 5: Сводка результатов всех экспериментов

| Эксперимент         | Конфигурация                                     | Val Acc (%) |
|---------------------|--------------------------------------------------|-------------|
| 1. Репараметризация | Train (multi-branch)                             | 93.88       |
|                     | Deploy (single-path, 3.7x быстрее)               | 93.88       |
| 2. Архитектуры      | RepVGG-A0 (deploy)                               | 93.88       |
|                     | VGG-16-BN                                        | 93.46       |
|                     | ResNet-20                                        | 91.39       |
| 3. Ablation         | Полная ( $3 \times 3 + 1 \times 1 + \text{id}$ ) | 93.88       |
|                     | Без identity                                     | 93.55       |
|                     | Без $1 \times 1$                                 | 93.29       |
|                     | Только $3 \times 3$                              | 93.38       |
| 4. Cutout           | RepVGG-A0 + Cutout                               | 94.90       |

Ограничения работы: (1) все эксперименты проведены на CIFAR-10 ( $32 \times 32$ ) - на ImageNet ( $224 \times 224$ ) выигрыш от простой deploy-архитектуры был бы значительнее

из-за большего объёма свёрточных вычислений; (2) обучение 100 эпох при стандарте 200–300 для CIFAR — при более длительном обучении ассигасу всех моделей была бы выше; (3) результаты получены из одного запуска с фиксированным seed, без оценки дисперсии по нескольким seed'ам.

Возможные направления развития: воспроизведение на ImageNet для оценки ускорения на реальных задачах, ablation по объёму данных (обучение на 10–50% выборки) для проверки устойчивости архитектуры, и сравнение с более поздними методами ре-параметризации (RepOptimizer, DBB).

## СПИСОК ИСТОЧНИКОВ

- [1] Ding X., Zhang X., Ma N., Han J., Ding G., Sun J. RepVGG: Making VGG-style ConvNets Great Again // Proceedings of the IEEE/CVF CVPR. – 2021. Режим доступа: <https://arxiv.org/abs/2101.03697> (дата обращения: 28.03.2026).
- [2] He K., Zhang X., Ren S., Sun J. Deep Residual Learning for Image Recognition // IEEE/CVF CVPR. – 2016.
- [3] Simonyan K., Zisserman A. Very Deep Convolutional Networks for Large-Scale Image Recognition // ICLR. – 2015.
- [4] DeVries T., Taylor G.W. Improved Regularization of Convolutional Neural Networks with Cutout // arXiv:1708.04552. – 2017.
- [5] Ioffe S., Szegedy C. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift // ICML. – 2015.

# ПРИЛОЖЕНИЕ

## А. Гиперпараметры обучения

Таблица 6: Полная конфигурация обучения (единая для всех моделей)

| Параметр                          | Значение                                                         |
|-----------------------------------|------------------------------------------------------------------|
| SGD: lr / momentum / weight decay | 0.1 / 0.9 / $10^{-4}$                                            |
| Scheduler                         | Cosine annealing, $\eta_{\min} = 10^{-6}$                        |
| Warmup                            | 5 эпох (линейный)                                                |
| Эпох / batch size                 | 100 / 128                                                        |
| Mixed precision                   | FP16 (torch.amp)                                                 |
| RandomCrop / HorizontalFlip       | $32 \times 32$ pad = 4 / $p = 0.5$                               |
| Cutout (эксп. 4)                  | $16 \times 16$                                                   |
| Нормализация                      | $\mu = (0.491, 0.482, 0.447)$ , $\sigma = (0.247, 0.243, 0.262)$ |

## Б. Архитектуры базовых моделей

Таблица 7: Базовые модели, адаптированные под CIFAR

| Модель        | Структура                                             | Парам. / FLOPs  |
|---------------|-------------------------------------------------------|-----------------|
| ResNet-20 [2] | BasicBlock $\times 3$ (16, 32, 64 кан.), GAP + FC     | 0.27M / 82.8M   |
| VGG-16-BN [3] | 13 conv $3 \times 3$ + BN (64 ... 512 кан.), GAP + FC | 14.72M / 629.1M |

## В. Исходный код и логи

Репозиторий: [https://github.com/Vlasikq/cv\\_repvgg](https://github.com/Vlasikq/cv_repvgg)

Логи экспериментов: <https://wandb.ai/vefulls-/RepVGG>